

Guide Technique SUPMEAL

Documentation technique complète

Projet individuel 4PROJ - Resit

SUPMEAL - Gestion de recettes, cookbooks partagés et planification

Bienvenue dans la documentation technique du projet SUPMEAL. Ce document regroupe l'architecture, la stack technique, la procédure d'installation, le déploiement et le fonctionnement interne des différents modules de l'application (Backend, Frontend et Base de données).

PRÉREQUIS

Avant d'installer SUPMEAL, les outils suivants doivent être installés sur la machine :

- Git : permet de cloner le dépôt du projet.
- Docker et Docker Compose : utilisés pour lancer le client web, l'API backend et PostgreSQL dans des conteneurs.
- Node.js et npm : nécessaires pour installer les dépendances ou exécuter les commandes de développement hors Docker.
- Un navigateur moderne : Firefox, Chrome ou Edge.
- Un éditeur de code, par exemple Visual Studio Code.

Le projet nécessite également un fichier `.env` à la racine. Un fichier `.env.example` indique les variables nécessaires sans exposer de données sensibles.

GitHub du projet :

<https://github.com/Keryann-MARTIN/SUPMEAL.git>

GUIDE DE DÉPLOIEMENT

Le déploiement de SUPMEAL repose sur Docker afin de simplifier l'exécution des trois briques imposées par le sujet : client web, serveur API et base de données.

1. Préparation de l'environnement

Les variables sensibles ou dépendantes de l'environnement sont notamment :

- Nom, utilisateur et mot de passe de la base PostgreSQL.
- Secret utilisé pour signer les tokens JWT.
- Identifiants OAuth GitHub.
- URL de callback OAuth.
- URL du frontend.

Élément	Description
POSTGRES_DB	Nom de la base PostgreSQL.
POSTGRES_USER	Utilisateur PostgreSQL.
POSTGRES_PASSWORD	Mot de passe PostgreSQL.
JWT_SECRET	Secret de signature des JWT.
GITHUB_CLIENT_ID	Client ID de l'application OAuth GitHub.
GITHUB_CLIENT_SECRET	Secret OAuth GitHub.
GITHUB_CALLBACK_URL	Callback OAuth, par défaut <code>http://localhost:3000/auth/github/callback</code> .
CLIENT_URL	URL du frontend, par défaut <code>http://localhost:5173</code> .

PROCÉDURE D'INSTALLATION

Étape 1 : Cloner le projet

```
git clone https://github.com/Keryann-MARTIN/SUPMEAL.git
cd SUPMEAL
```

Étape 2 : Configuration des variables d'environnement

Copier .env.example vers .env puis renseigner les valeurs locales.

```
Copy-Item .env.example .env
```

Structure attendue :

```
POSTGRES_DB=supmeal
POSTGRES_USER=supmeal
POSTGRES_PASSWORD=change_me
JWT_SECRET=change_me
GITHUB_CLIENT_ID=change_me
GITHUB_CLIENT_SECRET=change_me
GITHUB_CALLBACK_URL=http://localhost:3000/auth/github/callback
CLIENT_URL=http://localhost:5173
```

Étape 3 : Configuration OAuth GitHub

1. Créer une OAuth App depuis GitHub Developer settings.
2. Définir Homepage URL sur <http://localhost:5173>.
3. Définir Authorization callback URL sur <http://localhost:3000/auth/github/callback>.
4. Copier le Client ID et le Client Secret dans .env uniquement.

Étape 4 : Lancer la stack

```
docker compose up --build -d
```

Étape 5 : Vérifier les services

```
docker compose ps
```

```
Invoke-RestMethod http://localhost:3000/health
```

```
PS C:\Users\Keryann\Desktop\M.eng1\4PROJ-RESIT\supmeal> docker compose ps
NAME                IMAGE                COMMAND              SERVICE    CREATED    STATUS    PORTS
supmeal-client-1    supmeal-client      "docker-entrypoint.s..." client      3 days ago Up 3 days 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
supmeal-database-1 postgres:16-alpine   "docker-entrypoint.s..." database   3 weeks ago Up 13 days 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
supmeal-server-1    supmeal-server      "docker-entrypoint.s..." server     3 days ago Up 3 days 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\Users\Keryann\Desktop\M.eng1\4PROJ-RESIT\supmeal> |
```

JUSTIFICATION DES CHOIX TECHNOLOGIQUES

Les technologies utilisées dans SUPMEAL ont été choisies afin de rester cohérentes avec le sujet, d'assurer une application maintenable et de conserver une architecture compréhensible pour un projet individuel.

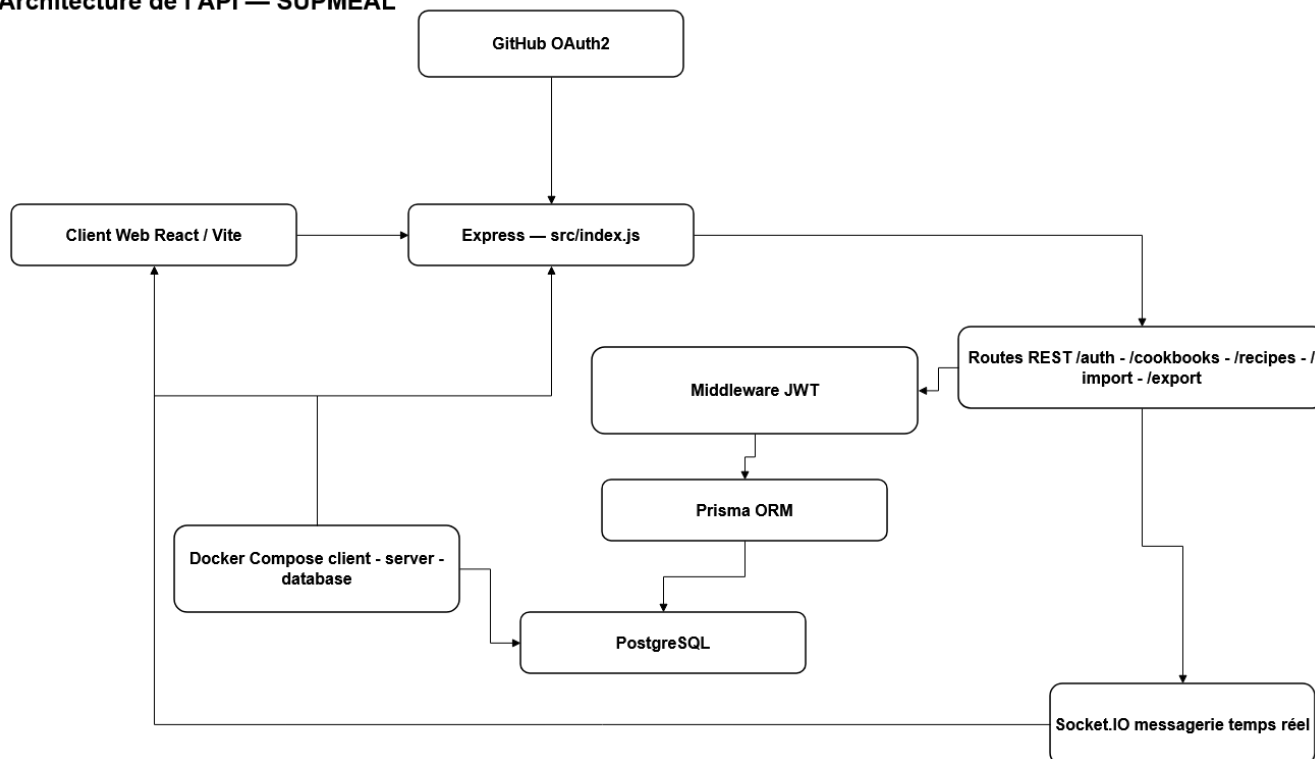
- React : interface web réactive organisée en pages.
- TypeScript : typage du frontend et des structures de données manipulées.
- Vite : environnement de développement rapide et léger.
- React Router DOM : routage de la SPA.
- Node.js / Express : API REST simple et flexible.
- PostgreSQL : base relationnelle robuste adaptée aux relations entre utilisateurs, cookbooks, recettes et messages.
- Prisma : ORM utilisé pour le schéma, les migrations et les accès PostgreSQL.
- JWT : sessions stateless pour les appels API protégés.
- bcryptjs : hachage des mots de passe locaux.
- OAuth2 GitHub : authentification externe sans demander le mot de passe GitHub à SUPMEAL.
- Socket.IO : messagerie mise à jour en temps réel.
- Multer : réception du fichier JSON lors d'un import.
- Docker : exécution reproductible du client, serveur et PostgreSQL.

PARTIE 1 : BACKEND (API)

1. Architecture Backend

Le backend de SUPMEAL est basé sur Express. L'application ne multiplie pas les couches artificielles : les routes regroupent les endpoints et la logique associée, le middleware JWT protège les accès, Prisma centralise l'accès aux données et PostgreSQL assure la persistance.

Architecture de l'API — SUPMEAL



1.1 Organisation en modules

Le répertoire server/ contient principalement :

- src/index.js : configuration Express, CORS, routes, serveur HTTP et Socket.IO.
- src/routes/auth.js : inscription, login, profil, mot de passe, préférences et GitHub OAuth.
- src/routes/cookbooks.js : cookbooks, membres, rôles et discussion.
- src/routes/recipes.js : recettes, filtres, favoris, planification et commentaires.
- src/routes/import.js : import JSON.
- src/routes/export.js : export JSON.
- src/middlewares/auth.js : middleware JWT.
- src/prisma.js : PrismaClient.
- prisma/schema.prisma : modèles de données.
- prisma/migrations/ : historique des migrations.

1.2 Flux de traitement d'une requête

5. Le client React envoie une requête à l'API.
6. Express sélectionne la route concernée.
7. Le middleware JWT vérifie le token si la route est protégée.
8. Le routeur contrôle les données et les permissions nécessaires.
9. Prisma interagit avec PostgreSQL.
10. La route renvoie une réponse JSON avec le code HTTP adapté.
11. Dans le cas de la messagerie, Socket.IO notifie les clients d'une mise à jour.

1.3 Avantages de l'architecture

- Séparation nette client / serveur / base de données.
- Accès aux données centralisé via Prisma.
- Permissions vérifiées côté serveur.
- Code court et lisible.
- Déploiement identique sur une nouvelle machine grâce à Docker.

2. Stack Technique Backend

2.1 Cœur de l'API

- Node.js 22 : environnement d'exécution JavaScript.
- Express : framework web utilisé pour les endpoints REST.
- cors : communication entre le frontend et l'API.
- dotenv : chargement des variables d'environnement.
- nodemon : redémarrage automatique en développement.

2.2 Base de données

- PostgreSQL 16.
- Prisma Client 6.19.3.
- Prisma Migrate pour versionner le schéma.

2.3 Authentification et sécurité

- jsonwebtoken : génération et vérification du JWT.
- bcryptjs : hachage des mots de passe.
- OAuth2 GitHub : connexion via fournisseur externe.
- Contrôles d'appartenance et de rôle pour les actions liées aux cookbooks.
- Aucun secret réel dans le dépôt.

2.4 Validation et utilitaires

- Validation manuelle des champs obligatoires et des valeurs attendues dans les routes.
- Multer : multipart/form-data pour l'import JSON.
- Socket.IO : communication temps réel.
- URLSearchParams : filtres et construction de l'URL OAuth.

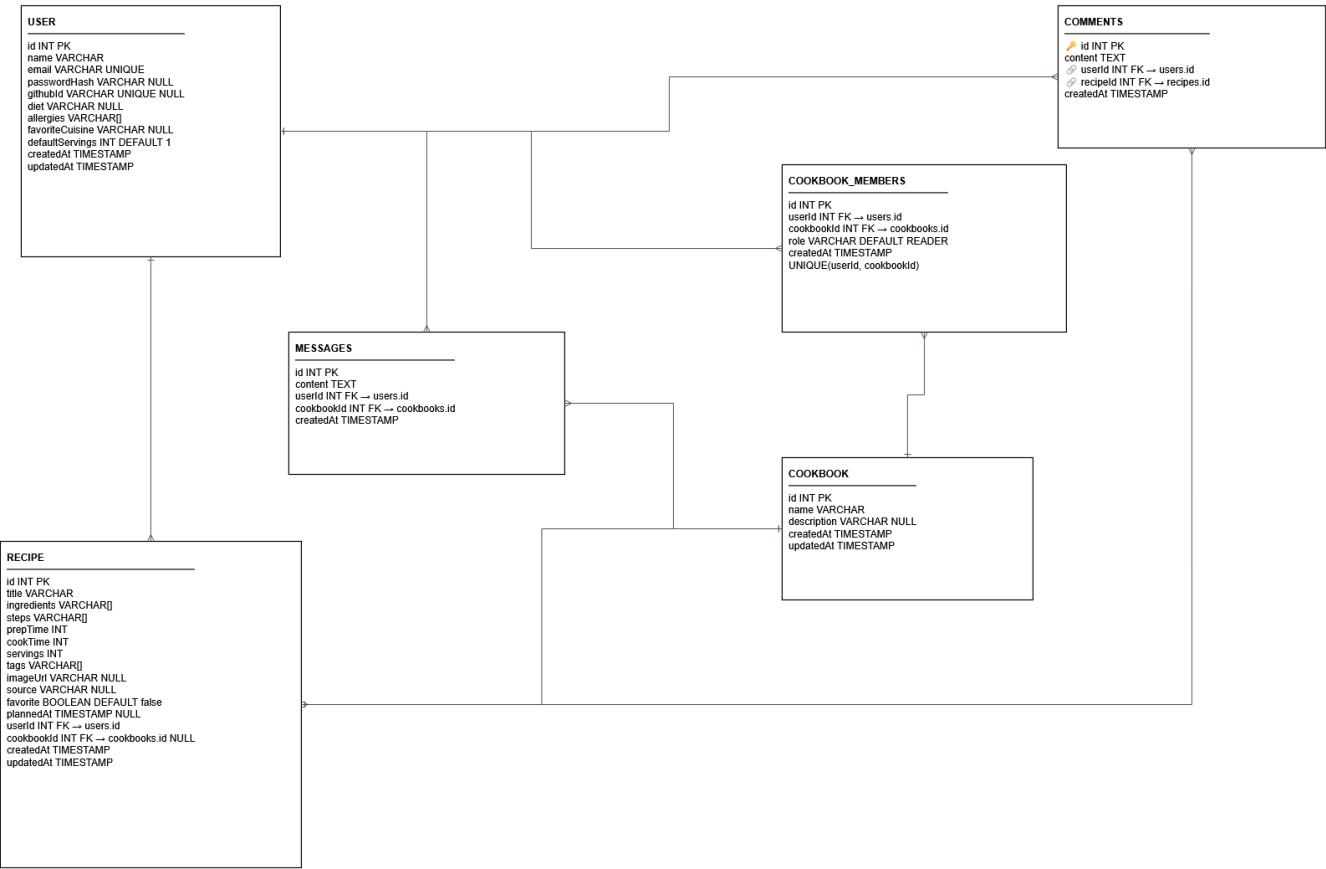
2.5 Gestion des erreurs

Les routes renvoient des codes 400 pour les données invalides, 401 pour l'authentification, 403 pour les permissions, 404 lorsqu'une ressource n'existe pas et 500 pour les erreurs serveur.

3. Base de Données et Modèles

SUPMEAL utilise PostgreSQL et Prisma. Le schéma est défini dans schema.prisma puis appliqué avec les migrations.

Schéma relationnel de la base de données — SUPMEAL



3.1 Connexion

```
const { PrismaClient } = require("@prisma/client")

const prisma = new PrismaClient()

module.exports = prisma
```

3.2 Modèles principaux

- User : compte, OAuth GitHub et préférences culinaires.
- Cookbook : livre partagé.
- CookbookMember : liaison utilisateur/cookbook avec rôle.
- Recipe : recette complète.
- Comment : commentaire sur une recette.
- Message : message d'un cookbook.

3.3 Modèle User

- name et email.
- passwordHash optionnel pour supporter les comptes OAuth.
- githubId optionnel et unique.
- diet, allergies, favoriteCuisine et defaultServings.
- relations vers recettes, commentaires, messages et appartenances.

3.4 Modèle Recipe

- title, ingredients et steps.
- prepTime, cookTime et servings.
- tags, imageUrl et source.
- favorite et plannedAt.
- userId obligatoire.
- cookbookId optionnel pour distinguer recette personnelle et recette partagée.

3.5 Relations et migrations

CookbookMember possède une contrainte unique sur le couple userId/cookbookId. Les suppressions en cascade sont utilisées sur plusieurs relations pour éviter les données orphelines. Les migrations Prisma conservent l'historique des évolutions du modèle.

4. Authentification et Sessions

4.1 Inscription et connexion locales

L'inscription requiert name, email et password. Le mot de passe doit contenir au moins 6 caractères. L'adresse email est normalisée en minuscules puis vérifiée pour éviter les doublons.

Lors de la connexion, bcrypt.compare vérifie le mot de passe. Un JWT signé avec JWT_SECRET est ensuite généré pour 7 jours.

4.2 Middleware JWT

Les routes protégées attendent Authorization: Bearer <token>. Le middleware vérifie le token puis définit req.userId.

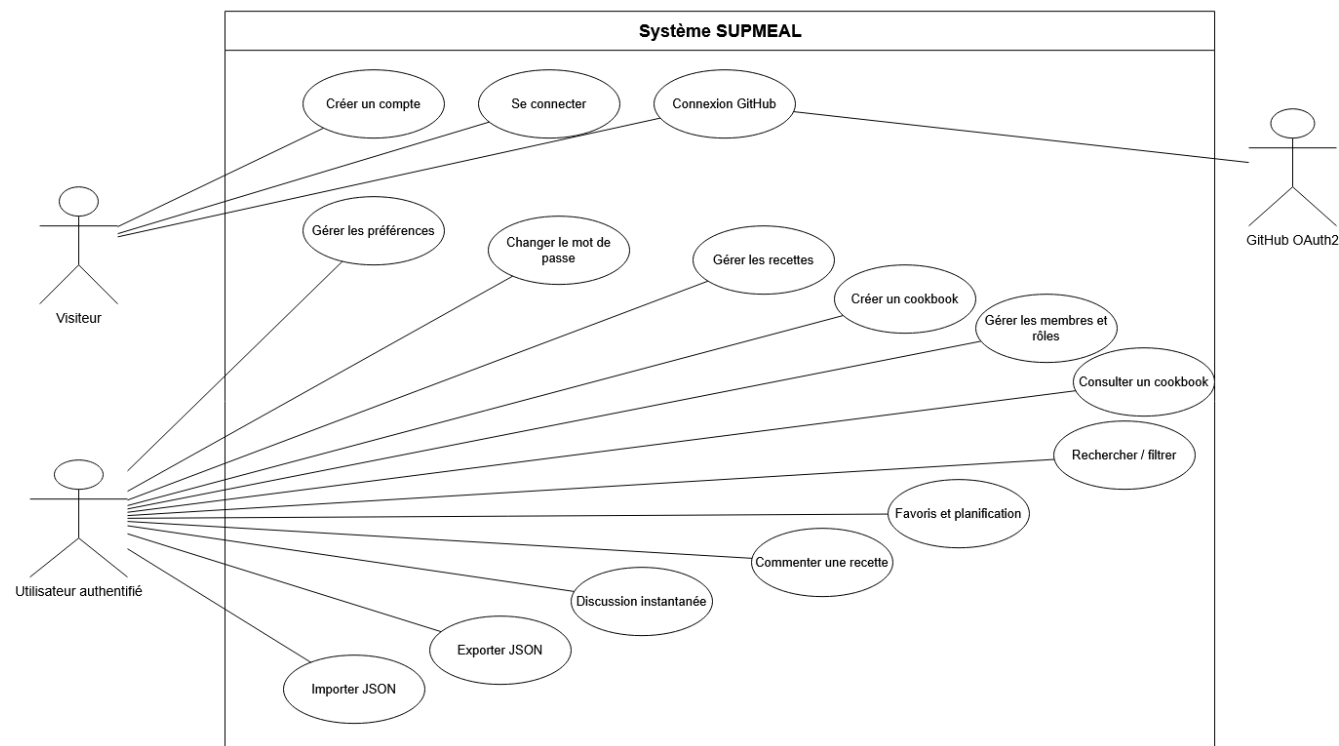
4.3 OAuth2 GitHub

Le flux OAuth redirige d'abord l'utilisateur vers GitHub. Le callback échange le code contre un access_token, récupère le profil et un email vérifié, puis associe githubId à un compte existant ou crée un nouveau compte.

4.4 Paramètres utilisateur

- Changement du mot de passe pour un compte local.
- Affichage spécifique pour un compte GitHub sans mot de passe SUPMEAL.
- Préférences culinaires persistées en base.
- État d'association GitHub visible dans l'interface.

Diagramme de cas d'utilisation — SUPMEAL



5. Endpoints de l'API

5.1 Authentification

Route	Protection	Rôle
POST /auth/register	Public	Créer un compte.
POST /auth/login	Public	Connexion locale.
GET /auth/me	JWT	Profil et préférences.
PATCH /auth/password	JWT	Modifier le mot de passe.
PATCH /auth/preferences	JWT	Modifier les préférences.
GET /auth/github	Public	Démarrer OAuth.
GET /auth/github/callback	Public	Callback OAuth.

5.2 Cookbooks

Route	Protection	Rôle
GET /cookbooks	JWT	Lister les cookbooks accessibles.
POST /cookbooks	JWT	Créer un cookbook.
GET /cookbooks/:id/members	JWT + membre	Lister les membres.
POST /cookbooks/:id/members	JWT + droits	Ajouter un membre.
PATCH /cookbooks/:id/members/:memberId	JWT + droits	Changer le rôle.
GET /cookbooks/:id/messages	JWT + membre	Lire la discussion.
POST /cookbooks/:id/messages	JWT + membre	Envoyer un message.

5.3 Recettes

Route	Protection	Rôle
GET /recipes	JWT	Lister/rechercher/filtrer.
POST /recipes	JWT + droits	Créer une recette.
PATCH /recipes/:id	JWT + accès	Modifier recette/favori/planning.
DELETE /recipes/:id	JWT + propriétaire	Supprimer.
GET /recipes/:id/comments	JWT + accès	Lire les commentaires.
POST /recipes/:id/comments	JWT + permission	Commenter.

5.4 Import, export et santé

Route	Protection	Rôle
GET /export	JWT	Exporter les données JSON.
POST /import	JWT	Importer un JSON SUPMEAL.
GET /health	Public	Vérifier API et PostgreSQL.

6. Gestion des Cookbooks et Permissions

Tout utilisateur peut créer un cookbook. Le créateur devient OWNER. Les autres membres sont ajoutés avec un rôle.

Rôle	Comportement
OWNER	Créateur/propriétaire du cookbook.
EDITOR	Membre pouvant participer à l'édition du contenu.
READER	Lecture du contenu ; l'ajout de recette est refusé.
COMMENTATOR	Consultation et commentaires selon les permissions.

6.1 Page dédiée

Chaque cookbook peut être ouvert via /cookbooks/:id. La page dédiée affiche son nom, sa description, le rôle courant, les recettes de ce cookbook et une recherche propre à cet espace.

6.2 Contrôle d'accès

Les contrôles de permissions sont effectués dans l'API. L'interface affiche les rôles mais ne décide jamais seule de l'accès.

7. Gestion des Recettes

7.1 Création et édition

- Titre, ingrédients, étapes.
- Temps de préparation/cuisson.
- Portions.
- Tags.
- Image optionnelle par URL.
- Source.
- Favori.
- Date de planification.
- Cookbook ou recette personnelle.

7.2 Recherche et filtrage

- Cookbook.
- Tags.
- Ingrédients.
- Temps de préparation.
- Temps de cuisson.
- Favoris.
- Recherche plein texte.

7.3 Favoris et planification

favorite est un booléen modifiable par PATCH. plannedAt contient une date optionnelle de planification du repas.

8. Commentaires et Messagerie

8.1 Commentaires

Les commentaires sont liés à une recette et à un utilisateur. L'API vérifie l'accès à la recette avant lecture ou écriture.

8.2 Messagerie instantanée

Les messages sont liés à un cookbook et persistés dans PostgreSQL. Socket.IO sert uniquement à signaler qu'une discussion a changé. Les clients rechargent ensuite les messages via l'API REST, ce qui conserve les contrôles d'accès existants.

8.3 Événement Socket.IO

```
io.emit("cookbook-messages-updated", { cookbookId })
```

9. Import et Export

9.1 Export JSON

L'export contient les recettes personnelles de l'utilisateur et les cookbooks dont il est OWNER, avec leurs recettes. Aucun passwordHash, token JWT ou secret n'est inclus.

9.2 Import JSON

Le fichier est reçu avec Multer, lu en mémoire, analysé avec JSON.parse puis validé. Une transaction Prisma crée les recettes personnelles et les cookbooks. L'utilisateur courant devient OWNER des cookbooks importés.

PARTIE 2 : FRONTEND (APPLICATION WEB)

1. Architecture Frontend

Le frontend est une SPA React/Vite. Il n'effectue pas d'accès direct à PostgreSQL et communique uniquement avec l'API.

1.1 Structure organisationnelle

- App.tsx : routage.
- pages/Home.tsx : dashboard, cookbooks, membres, discussion, import/export.
- pages/Recipes.tsx : recettes et filtres.
- pages/Cookbook.tsx : page dédiée d'un cookbook.
- pages/Login.tsx et Register.tsx : authentification locale.
- pages/Settings.tsx : préférences et sécurité du compte.
- pages/OAuthCallback.tsx : finalisation OAuth.
- socket.ts : client Socket.IO.
- App.css : styles.

1.2 Cycle d'une page React

Les pages utilisent useState pour l'état local, useEffect pour les chargements initiaux et useCallback lorsque la stabilité d'une fonction asynchrone est nécessaire.

1.3 Routing

- / - accueil.
- /recipes - recettes.
- /cookbooks/:id - cookbook dédié.
- /settings - paramètres.
- /login et /register - authentification.
- /oauth/callback - callback frontend OAuth.

2. Stack Technique Frontend

- React et React DOM.
- TypeScript.
- Vite.
- React Router DOM.
- socket.io-client.
- CSS classique.

3. Gestion de l'état

SUPMEAL ne nécessite pas de Redux ou Context API. Les états sont localisés dans les pages. Le JWT est conservé dans localStorage et transmis dans l'en-tête Authorization.

4. Services et API Fetching

Les appels utilisent fetch directement depuis les pages. Les paramètres de recherche sont construits avec URLSearchParams. Les formulaires de création/édition convertissent les champs texte en tableaux lorsque nécessaire.

5. Pages clés

- Home : cookbooks et collaboration.
- Recipes : formulaire et liste globale.
- Cookbook : recherche limitée à un cookbook.
- Settings : préférences et authentification.
- OAuthCallback : stockage du JWT puis redirection.

6. UX et responsive

L'interface utilise des cartes, boutons cohérents, formulaires explicites et quelques media queries. Le choix a été fait de conserver une interface simple plutôt qu'un framework CSS complet.

PARTIE 3 : CONTENEURISATION ET EXPLOITATION

1. Architecture Docker

Élément	Description
client	React/Vite - port 5173.
server	Node/Express - port 3000.
database	PostgreSQL 16 - port 5432.

1.1 Dockerfile client

Image node:22-alpine, installation avec npm ci, exposition du port 5173 et lancement de Vite avec --host 0.0.0.0.

1.2 Dockerfile serveur

Image node:22-alpine, installation avec npm ci, génération de Prisma Client puis npm run dev.

1.3 Volume PostgreSQL

Le volume postgres_data conserve la base malgré l'arrêt ou la recréation des conteneurs.

2. Migrations Prisma

Les migrations présentes dans server/prisma/migrations doivent être conservées dans le rendu. Elles permettent de retrouver le schéma de la base sur une nouvelle installation.

3. Vérification de fonctionnement

```
docker compose down
docker compose up --build -d
docker compose ps
Invoke-RestMethod http://localhost:3000/health
```

4. Sécurité du rendu

- .env non commité.
- .env.example sans secrets.
- GITHUB_CLIENT_SECRET et JWT_SECRET absents du code source.
- Mots de passe hashés.
- Migrations Prisma incluses.

5. Dépannage courant

- Si Prisma ne joint pas database:5432, vérifier que le service database est Up.
- Si une nouvelle dépendance n'existe pas dans le conteneur, reconstruire avec docker compose up --build -d.
- Si OAuth échoue, vérifier les variables GitHub et la callback URL.
- Si le frontend semble ancien, reconstruire le service client.